# Claude GUI: development plan for a web-based IDE wrapping Claude CLI

**Claude GUI is technically feasible and well-supported by mature open-source tooling.** The architecture combines a custom Next.js server with WebSocket-based terminal emulation, the official Claude Agent SDK for programmatic CLI control, and a three-panel IDE layout using battle-tested React libraries. Several community projects—notably `claudecodeui` and `claude-code-web`—already demonstrate this pattern, validating the approach. This document covers every technical layer: from spawning Claude Code processes to rendering live HTML presentations in the browser.

---

## 1. Claude CLI integration and programmatic control

Claude Code (formerly Claude CLI) is Anthropic's agentic coding tool installed via native installer (`curl -fsSL https://claude.ai/install.sh | bash`) or Homebrew. It requires macOS 13+, Windows 10+, or Ubuntu 20.04+, with **4 GB+ RAM** and a Pro/Max/Team/Enterprise subscription or API key. The CLI provides a rich set of built-in tools—**Read, Write, Edit, Bash, GlobTool, GrepTool, LS, WebSearch, WebFetch**—and supports MCP (Model Context Protocol) for extensibility. Claude

### The Agent SDK is the recommended programmatic interface

Rather than raw `child_process.spawn()` (which has documented hanging issues when spawning Claude Code from Node.js), GitHub Anthropic provides the **Claude Agent SDK** (`@anthropic-ai/claude-agent-sdk`). This TypeScript package internally manages subprocess lifecycle, error recovery, and message parsing:

```typescript
import { query } from '@anthropic-ai/claude-agent-sdk';

for await (const message of query({
  prompt: 'Find and fix bugs in auth.py',
  options: {
    allowedTools: ['Read', 'Edit', 'Bash'],
    permissionMode: 'bypassPermissions',
    model: 'sonnet',
    maxTurns: 10,
  },
})) {
  // Handle SDKAssistantMessage, SDKResultMessage, etc.
}
```

The SDK provides `query()` as an async generator yielding typed `SDKMessage` events (assistant, user, result, system, partial, status), plus `startup()` for pre-warming the subprocess (~20x faster first query), and `createSdkMcpServer()` for in-process MCP server creation. The `SDKResultMessage` includes `total_cost_usd`, `usage` token counts, `session_id`, and optional `structured_output` when using `--json-schema`.

### CLI output formats power the streaming architecture

For direct CLI usage, three output modes matter for the GUI:

- **`--output-format stream-json`**: Newline-delimited JSON (NDJSON) emitted in real-time, GitHub including `assistant` messages, `stream_event` deltas (with `--verbose --include-partial-messages`), and a final `result` message with cost/usage data.
- **`--input-format stream-json`**: Accepts NDJSON on stdin for bidirectional multi-turn conversations.
- **`--json-schema`**: Returns structured JSON matching a provided schema in the `structured_output` field.

Key CLI flags for server integration include `--bare` (skip hooks/plugins for faster startup), `--max-turns` and `--max-budget-usd` (prevent runaway processes), `--no-session-persistence` (stateless operation), and `--dangerously-skip-permissions` (automated workflows). Session management supports resume by ID (`-r`), continuation (`-c`), forking (`--fork-session`), and named sessions (`-n`).

### Conversation and session management

Sessions persist to `~/.claude/projects/` on disk. The GUI should read this directory structure to display session history, enable resume/fork, and show cost tracking. The `/compact` command and automatic context compaction handle long conversations. Custom slash commands from `.claude/commands/` directories Shipyard and custom subagents from `.claude/agents/` can extend functionality.

---

## 2. Three-panel IDE layout with React and Next.js

The layout follows the standard IDE pattern: file explorer (left), code editor (center), and preview panel (right), with an optional bottom terminal panel. Two libraries form the structural foundation.

### react-resizable-panels v4 delivers the panel system

With **5.2k GitHub stars**, **2.76M weekly npm downloads**, and adoption by shadcn/ui and OpenAI, `react-resizable-panels` (by bvaughn) is the clear choice. Version 4 (released December 2025) GitHub added pixel-based units and improved accessibility. The library provides three core components—`PanelGroup`, `Panel`, and `PanelResizeHandle`—with built-in

collapse/expand (Readmex) via the `collapsible` prop and automatic layout persistence via `autoSaveId`:

```tsx
<PanelGroup direction="horizontal" autoSaveId="claude-gui-layout">
  <Panel defaultSize={20} minSize={15} collapsible collapsedSize={4}>
    <FileExplorer />
  </Panel>
  <PanelResizeHandle />
  <Panel defaultSize={50} minSize={30}>
    <PanelGroup direction="vertical" autoSaveId="editor-layout">
      <Panel defaultSize={70}><EditorTabs /></Panel>
      <PanelResizeHandle />
      <Panel defaultSize={30} collapsible><TerminalPanel /></Panel>
    </PanelGroup>
  </Panel>
  <PanelResizeHandle />
  <Panel defaultSize={30} collapsible collapsedSize={0}>
    <PreviewPanel />
  </Panel>
</PanelGroup>
```

The `collapsedSize` prop controls the collapsed width (e.g., `4` for an icon-only sidebar, `0` for fully hidden). Imperative methods `panelRef.current.collapse()` and `.expand()` enable programmatic fold/unfold. Panel sizes auto-save to localStorage, and nested `PanelGroup` components handle the vertical editor/terminal split. The library is SSR-compatible and WAI-ARIA compliant.

**Alternatives considered:** `allotment` (~1.2k stars) derives from VS Code's split view codebase (GitHub) but lacks SSR support and requires `next/dynamic` with `ssr: false`. (GitHub) `react-split-pane` (~3.3k stars) is effectively unmaintained. Neither matches react-resizable-panels in ecosystem support or maintenance velocity.

### Monaco Editor provides VS Code-grade editing

The `@monaco-editor/react` package wraps Monaco Editor with zero webpack configuration—(GitHub) it loads from CDN by default. In Next.js App Router, a `"use client"` directive suffices; (Deno) for Pages Router, use `next/dynamic` with `ssr: false` since Monaco requires browser APIs. (swyx)

Monaco's model system enables multi-file/tab support: each file gets a model keyed by its `path` prop, preserving cursor position, scroll state, and undo history when switching tabs. (GitHub) The editor supports **100+ languages**, IntelliSense, minimap, code folding, multiple cursors, and custom themes. (NPM Compare) Bundle size is **5–10 MB** uncompressed (Agenthicks) but loads once and caches when using the CDN approach.

**CodeMirror 6** is the alternative if bundle size is critical (~300 KB core) (Agenthicks) or mobile support is needed. It uses a modular, extension-based architecture and powers Replit (Replit) and Chrome DevTools. (Sourcegraph) However, Monaco's out-of-the-box IntelliSense and VS Code familiarity make it the stronger choice for a full IDE experience.

### react-arborist handles the file tree

For the left panel file/folder explorer, `react-arborist` (by brimdata, v3.4.x) provides virtualized rendering (via react-window for thousands of items), built-in drag-and-drop, inline renaming (OpenReplay) (F2), and multi-selection. Custom node rendering allows adding file-type icons mapped from extensions (e.g., `.ts` → TypeScript icon). For context menus, `@radix-ui/react-context-menu` integrates cleanly.

### State management with Zustand

**Zustand** (v5.x) is recommended as the primary state store (DEV Community) because it works outside React components—critical for updating state from WebSocket handlers. The store should track panel sizes, open files/tabs, active file, editor states per file (cursor, scroll, selections), sidebar visibility, and terminal sessions. The `persist` middleware handles localStorage serialization. For rapidly changing data like terminal buffers, **Jotai** atoms can complement Zustand with fine-grained reactivity. (OpenReplay)

A **command palette** using `cmdk` (by Paco Coursey, powers Linear and Vercel) (Shadcn) provides ⌘K / Ctrl+Shift+P functionality. (LogRocket) Essential keyboard shortcuts include ⌘P (quick file open), ⌘B (toggle sidebar), ⌘J (toggle terminal), and ⌘S (save).

---

## 3. Real-time preview for HTML, PDF, Markdown, and images

The right panel serves as a universal preview surface, detecting file type and rendering accordingly.

### HTML preview uses sandboxed iframes

The `srcdoc` attribute is the recommended approach—setting `iframe.srcdoc = htmlString` triggers immediate rendering without network requests. For security, apply `sandbox="allow-scripts"` (never combine `allow-scripts` with `allow-same-origin` on same-origin content, as the iframe could remove its own sandbox). (TheLinuxCode) Additional protection comes from the `csp` iframe attribute in Chromium (MDN Web Docs) and `referrerpolicy="no-referrer"`.

For live updates without full iframe refresh, **debounced srcdoc updates** (300–500 ms) are the simplest pattern. For CSS-only changes, send updated styles via `postMessage` and patch the `<style>` element inside the iframe. Parent-iframe communication via `postMessage` also enables forwarding `console.log` output and error reporting back to the main UI. (Medium)

### PDF viewing with react-pdf v10

The `react-pdf` package (by wojtekmaj, based on pdf.js Nutrient 5.x) provides `<Document>` and `<Page>` components with page-by-page navigation. Track `numPages` from `onLoadSuccess`, maintain a `pageNumber` state, and render the current page. The `<Thumbnail>` component enables a clickable page sidebar. GitHub PDF.js uses Web Workers for background parsing, so the worker file must be configured. npm For PDF *generation* from HTML, **Puppeteer** (server-side) provides the highest fidelity; IronPDF **html2pdf.js** works for quick client-side exports. GitHub Nutrient

### Markdown rendering pipeline

**react-markdown** (v10) transforms Markdown → remark AST → rehype AST → React components with no `dangerouslySetInnerHTML`. Strapi Key plugins include `remark-gfm` (tables, task lists), `remark-math` + `rehype-katex` (LaTeX), and `rehype-highlight` Strapi or `rehype-pretty-code` (syntax highlighting). For syntax highlighting, **Shiki** provides VS Code-quality output but is heavier (~280 KB + WASM); **Prism.js** (~10–30 KB) is faster for real-time client-side preview.

### Image preview with zoom and pan

Native browser support covers JPEG, PNG, GIF, WebP, SVG, and AVIF. For zoom/pan interaction, `react-zoom-pan-pinch` handles pinch-to-zoom and mouse wheel zoom on any element. SVGs can render inline for full CSS styling and interaction, with `react-svg-pan-zoom` for dedicated SVG navigation. GitHub

### Page-by-page navigation pattern

A universal navigation component serves PDFs, presentations, and multi-page documents: Previous/Next buttons, a page indicator ("Page 3 of 12"), direct page input, and keyboard shortcuts (arrow keys, Page Up/Down). A thumbnail sidebar with Intersection Observer-based current-page tracking works for both PDF pages and presentation slides.

---

## 4. HTML presentations with reveal.js and conversational editing

### reveal.js is the presentation engine

With **70k+ GitHub stars**, reveal.js is the dominant HTML presentation framework. Slides are `<section>` elements supporting nested (vertical) slides, Markdown content, Auto-Animate transitions, speaker notes, LaTeX rendering, and built-in syntax highlighting. The framework ships with **12 themes** and **6 transition types**.

The critical API methods for programmatic control are:

- **`Reveal.sync()`**: Re-reads the DOM after dynamically adding/removing slides—* slides essential* for real-time editing.

- `Reveal.slide(h, v, f)`: Navigate to a specific slide by index. (Reveal.js)
- `Reveal.configure()`: Update settings at runtime (e.g., `autoSlide`, theme). (slides)
- **postMessage API**: When running in an iframe with `postMessage: true` and `postMessageEvents: true`, the parent can control navigation and receive `slidechanged` events. (slides)

### Architecture for slide-by-slide creation

The recommended data model stores slides as a JSON array: `[{ id, html, css, notes, transition, background }]`. The editor panel shows per-slide HTML/Markdown. The preview panel runs reveal.js in a sandboxed iframe. On each edit, the parent sends updated slide HTML via `postMessage`; a script inside the iframe patches the specific `<section>` element's innerHTML and calls `Reveal.sync()`. This avoids full iframe reload and preserves the current slide position.

For **AI-powered conversational editing**, Claude receives the current slide's HTML plus the user's natural language instruction ("Make the title bigger on slide 3," "Add a bullet list about Q2 revenue") and returns modified HTML. The frontend applies the diff to the specific slide, calls `Reveal.sync()`, and the preview updates instantly. Using `--json-schema` with the Agent SDK ensures Claude returns structured slide data that can be reliably parsed.

### Export capabilities

For PPTX export, **PptxGenJS** creates native `.pptx` files in browser or Node.js with support for text, tables, shapes, images, charts, and slide masters. For PDF export, **DeckTape** (Node.js, Puppeteer-based) captures each slide individually with excellent fidelity. (GitHub) The reveal.js `?print-pdf` query parameter with Chrome's print dialog works for simpler PDF needs. (Reveal.js) (GitHub)

**Alternatives**: Marp converts Markdown to slides (Marp) (good for developer workflows but less suited for real-time editing). impress.js offers Prezi-style 3D navigation but lacks themes, plugins, and active maintenance.

---

## 5. WebSocket and SSE for real-time streaming

### Next.js requires a custom server for WebSocket

Next.js does **not** natively support WebSocket connections— (fly) the built-in server intercepts HTTP upgrade events. (GitHub) (Pedro Alonso) The standard solution is a custom `server.js` using the `ws` library: (fly)

```javascript
```

```
const { WebSocketServer } = require('ws');
const next = require('next');
const http = require('http');

const nextApp = next({ dev: process.env.NODE_ENV !== 'production' });
const server = http.createServer(nextApp.getRequestHandler());
const wss = new WebSocketServer({ noServer: true });

server.on('upgrade', (req, socket, head) => {
  const { pathname } = new URL(req.url, 'http://localhost');
  if (pathname === '/_next/webpack-hmr') {
    nextApp.getUpgradeHandler()(req, socket, head); // Preserve HMR
  } else if (pathname.startsWith('/ws/')) {
    wss.handleUpgrade(req, socket, head, (ws) => wss.emit('connection', ws, req));
  }
});
```

**`ws` is preferred over `socket.io`** for this use case: lower latency (~5 KB vs ~10.4 KB), standard WebSocket protocol (any client works), and no overhead from rooms/namespaces. (Generalist Programmer) code-server uses `ws` v8.14.2 for the same reason. (DeepWiki) The custom server disables Automatic Static Optimization and requires Node.js-based hosting (Docker, Railway, Fly.io)—**cannot deploy to Vercel.** (fly) (Socket.IO)

### SSE works for one-way streaming without a custom server

Server-Sent Events work natively in Next.js App Router using `ReadableStream` or `TransformStream`. (Pedro Alonso) SSE suits read-only streaming scenarios: Claude response tokens, progress notifications, log tailing. The browser's `EventSource` API provides built-in reconnection with exponential backoff. (DEV Community) However, **terminal emulation requires WebSocket** for bidirectional input/output.

### Connection management pattern

A `WebSocketManager` class should handle reconnection with exponential backoff (starting at 1 second, capping at 30 seconds), heartbeat pings every 29 seconds (under typical 30-second proxy timeouts), and multiplexing multiple channels (terminal, file watch, Claude output) over a single connection using message-type framing.

---

## 6. Terminal emulation with xterm.js and node-pty

The universal pattern used by VS Code, code-server, Theia, Gitpod, and Replit is: **xterm.js (browser) ↔ WebSocket ↔ node-pty (server).** (Sabuj Kundu) This is the proven architecture for Claude GUI's embedded terminal.

### Server-side: node-pty spawns pseudo-terminals

`node-pty` (maintained by Microsoft, used by VS Code) forks proper PTYs, unlike `child_process` which doesn't maintain persistent interactive sessions. Eddy Mens PTY output streams raw ANSI escape codes, cursor movement, and color data that xterm.js renders faithfully: PyPI

```javascript
const ptyProcess = pty.spawn('bash', [], {
  name: 'xterm-256color', cols: 80, rows: 24,
  cwd: workspaceDir, env: process.env
});
ptyProcess.onData(data => ws.send(data));
ws.on('message', data => ptyProcess.write(data));
```

Note that `node-pty` requires native C++ compilation at build time—prebuilt binaries cover common platforms, but Docker deployment simplifies this.

### Client-side: xterm.js with addons

The `@xterm/xterm` package (v5.x, scoped) is the **de facto standard** SourceForge with 17k+ GitHub stars. Essential addons include `@xterm/addon-fit` (auto-resize to container), `@xterm/addon-webgl` (GPU-accelerated rendering for fast output), `@xterm/addon-search` (buffer search), and `@xterm/addon-web-links` (clickable URLs). GitHub For React integration, `react-xtermjs` by Qovery provides a hooks-based wrapper. Qovery

### Backpressure is critical for production

xterm.js has a hardcoded **50 MB write buffer limit**—data beyond this is silently discarded. Fast producers (several GB/s from commands like `cat /dev/urandom`) overwhelm xterm.js throughput (5–35 MB/s). Xterm.js Implement watermark-based flow control: pause the PTY when the buffer exceeds a high watermark (100 KB), resume when it drops below a low watermark (10 KB), using xterm.js's write callback to track drain. Xterm.js

---

## 7. Secure file system access from Next.js API routes

Node.js `fs` works in both App Router and Pages Router API routes since they execute server-side. The primary threat is **path traversal**—Next.js provides no built-in protection for custom file operations. GitHub

### Path sanitization is non-negotiable

Every file operation must resolve the user-supplied path against a base directory and verify the result stays within bounds:

```typescript
function resolveSafe(rootDir: string, userPath: string): string {
  const resolved = path.resolve(rootDir, path.normalize(userPath).replace(/\0/g, '')
  if (!resolved.startsWith(path.resolve(rootDir) + path.sep) && resolved !== path.re
    throw new Error('Path traversal attempt');
  }
  return resolved;
}
```

Additional measures include denying access to dotfiles (`.env`, `.git`), checking for symlinks pointing outside the sandbox via `fs.lstat()`, enforcing file size limits, rate-limiting operations, and running the server with minimal OS permissions.

### File watching with chokidar

**Chokidar v5** (ESM-only, Node 20+ required) npm watches the workspace directory and broadcasts changes via WebSocket to all connected clients. Configure `ignoreInitial: true`, ignore dotfiles, set a reasonable `depth` limit, and use `awaitWriteFinish` for debouncing rapid writes. This enables the file explorer to reflect external changes (e.g., Claude CLI editing files) in real-time.

### REST API design for file operations

A clean REST API exposes file CRUD:

- `GET /api/files?path=/src` → directory listing
- `GET /api/files/read?path=/src/app.ts` → file content
- `POST /api/files/write` → create/update `{ path, content }`
- `DELETE /api/files?path=/src/old.ts` → delete
- `POST /api/files/mkdir` → create directory
- `POST /api/files/rename` → rename/move `{ oldPath, newPath }`
- `GET /api/files/stat?path=...` → metadata (size, mtime, type)
- WebSocket `/ws/files` → real-time change notifications

---

## 8. Existing projects that validate this architecture

Several open-source projects already wrap Claude CLI in web GUIs, confirming the pattern's viability and providing reference implementations.

### Claude-specific web UIs

**CloudCLI / claudecodeui** (siteboon/claudecodeui) is the most feature-complete: Threads

GitHub React + Vite + Express + WebSocket, with session auto-discovery from `~/.claude/projects/`, file tree, Git explorer, terminal, MCP server management, and plugin system. GitHub **claude-code-web** (vultuk) provides a clean Express + WebSocket bridge with CodeMirror editor and multi-session support. GitHub **claude-code-webui** (sugyan) offers a lightweight, npm-installable wrapper. GitHub **Claudito** demonstrates advanced patterns: Slack integration, autonomous building loops, and named shell commands per project. GitHub

### Web IDE reference architectures

**code-server** (~70k stars) is the gold standard for wrapping a desktop app as a web server— Coderit patches VS Code's Electron layer to serve via HTTP, Code Server uses `node-pty` + `ws` for terminals. DeepWiki **Eclipse Theia** (~20k stars) provides a modular, extensible IDE framework with dependency injection, LSP support, and both browser and desktop deployment from a single codebase— DZoneits Theia AI framework even includes Claude Code integration. **Bolt.new/bolt.diy** by StackBlitz runs Node.js entirely in-browser via WebContainers (WebAssembly), demonstrating an alternative to server-side execution. GitHub

### The architectural shift toward agent orchestration

A notable trend across these projects: the primary interface is shifting from "code editor" to **"agent management console."** Cursor 3's "Glass" redesign puts agent orchestration at the center. The New Stack Continue.dev's Mission Control automates workflows with cron and webhooks. Claudito runs autonomous building loops. Claude GUI should embrace this pattern—the editor and preview are tools *in service of* the Claude agent, not the other way around.

---

## 9. Recommended technology stack and architecture summary

The complete stack for Claude GUI, organized by layer:

| Layer | Technology | Purpose |
|---|---|---|
| Framework | Next.js 14+ (App Router) + custom `server.js` | SSR, routing, API routes |
| Panels | `react-resizable-panels` v4 | Resizable, collapsible layout |
| Editor | `@monaco-editor/react` + `monaco-editor` | Code editing with IntelliSense |
| File tree | `react-arborist` v3.4 | Virtualized file explorer |
| Terminal | `@xterm/xterm` v5 + `react-xtermjs` | Browser terminal |
| Terminal backend | `node-pty` | Server-side PTY |
| WebSocket | `ws` | Real-time bidirectional |
| Claude integration | `@anthropic-ai/claude-agent-sdk` | Programmatic Claude Code control |
| State | `zustand` v5 (+ `jotai` optional) | App state with persistence |
| HTML preview | iframe + `srcdoc` + `sandbox` | Sandboxed rendering |
| PDF viewer | `react-pdf` v10 | Page-by-page PDF display |
| Markdown | `react-markdown` + `remark-gfm` + `rehype-highlight` | Rich Markdown rendering |
| Presentations | `reveal.js` 5.x | Slide authoring and preview |
| File watching | `chokidar` v5 | Real-time filesystem events |
| Command palette | `cmdk` (via shadcn/ui) | ⌘K interface |
| UI primitives | shadcn/ui + Radix + Tailwind CSS | Consistent, accessible UI |
| PPTX export | `PptxGenJS` | Native PowerPoint generation |

## High-level architecture

```
┌──────────────────────────────────────────────────────────────────┐
│  Browser (Chrome)                                                 │
│  ┌──────────┬──────────────────────┬──────────────────┐          │
│  │ File     │ Monaco Editor        │ Preview Panel    │          │
│  │ Explorer │ (multi-tab)          │ (HTML/PDF/MD/    │          │
```

```
│  │ (react-  │                        │  Slides/Image)    │  │
│  │ arborist)│                        │                   │  │
│  ├──────────┴────────────────────────┤                   │  │
│  │ Terminal (xterm.js)               │                   │  │
│  │ + Claude Chat Panel               │                   │  │
│  └───────────────────────────────────┴───────────────────┘  │
│           ↕ WebSocket          ↕ HTTP/SSE                   │
└────────────┼──────────────────────┼────────────────────────┘
             ↕                      ↕

┌──────────────────────────────────────────────────────────┐
│  Custom Node.js Server (server.js)                        │
│  ├── Next.js request handler (pages, API routes)          │
│  ├── WebSocket server (ws)                                │
│  │   ├── /ws/terminal → node-pty sessions                 │
│  │   ├── /ws/claude → Agent SDK streaming                 │
│  │   └── /ws/files → chokidar file notifications          │
│  ├── REST API (/api/files/*) → fs module (sandboxed)      │
│  └── Claude Agent SDK → Claude Code subprocess            │
└──────────────────────────────────────────────────────────┘
```

## Conclusion

The Claude GUI project sits at the convergence of proven web IDE patterns and the emerging agent-orchestration paradigm. The technical risk is low: every major component—terminal emulation, resizable panels, code editing, file system access, real-time streaming—has mature, production-tested libraries with clear integration paths. The **Claude Agent SDK** eliminates the historically painful step of raw subprocess management, and **react-resizable-panels** + **Monaco Editor** + **xterm.js** form a battle-tested foundation used by the largest web IDE products.

The most important architectural decision is using a **custom Next.js server** rather than standard Next.js deployment—this unlocks WebSocket support for terminal and Claude streaming but constrains deployment to Node.js hosts (ruling out Vercel serverless). Socket.IO The second key decision is treating the **Agent SDK as the primary integration point** rather than raw CLI spawning, gaining typed messages, error recovery, and session management for free.

Where this project can differentiate is in the **preview and presentation layer**: real-time HTML/slide rendering with reveal.js, conversational AI editing of visual content, and seamless export to PDF/PPTX. No existing Claude web UI combines a full IDE layout with rich preview capabilities. The shift toward agent-first interfaces—where the GUI orchestrates Claude rather than merely displaying its output—represents the highest-leverage design direction. The New Stack